피드포워드 네트워크 (Feed-Forward Network) 구현

개요

트랜스포머의 피드포워드 네트워크(FFN)는 각 위치에서 독립적으로 동작하는 완전 연결 신경망입니다. 어텐션 메커니즘이 시퀀스 내 토큰 간의 관계를 모델링한다면, 피드포워드 네트워크는 각 토큰의 표현을 개별적으로 변환하여 모델의 표현력을 크게 향상시킵니다.

포지션별 피드포워드 네트워크의 역할과 구조

1. 기본 개념과 필요성

어텐션과의 상호보완적 역할

```
# 어텐션: 토큰 간 상호작용 (토큰 간 정보 교환)
attention_output = attention(query, key, value) # 시퀀스 내 관계 모델링

# 피드포워드: 각 토큰의 개별적 변환 (토큰별 독립적 처리)
ff_output = feedforward(attention_output) # 위치별 표현 변환
```

포지션별(Position-wise) 처리의 의미

"포지션별"이라는 용어는 네트워크가 시퀀스의 각 위치에서 **동일한 변환**을 **독립적으로** 적용한다는 의미입니다:

```
# 시퀀스의 각 위치에 동일한 FFN 적용
for position in range(sequence_length):
output[position] = FFN(input[position])
# 이는 다음과 같이 병렬로 처리됩니다:
# [batch_size, seq_len, d_model] → [batch_size, seq_len, d_model]
```

2. 피드포워드 네트워크의 구조

기본 아키텍처

```
입력 (d_model) → 선형변환1 (d_ff) → ReLU → 드롭아웃 → 선형변환2 (d_model) → 출력
```

수식으로 표현하면:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

여기서:

- W_1 : [d_model, d_ff] 크기의 가중치 행렬
- W_2 : [d_ff, d_model] 크기의 가중치 행렬
- b_1 , b_2 : 편향(bias) 벡터
- $\max(0, \cdot)$: ReLU 활성화 함수

두 개의 선형 변환과 ReLU 활성화 함수 분석

1. 첫 번째 선형 변환: 차원 확장

```
def  first_linear_transformation():
"""
첫 번째 선형 변환의 역할과 효과
d_model → d_ff (보통 4배 확장)
예: 512 → 2048
"""
transformation_effects = {
"차원 확장": {
"목적": "표현 공간 확대",
"효과": "더 풍부한 특징 표현 가능",
"비유": "좁은 방에서 넓은 방으로 이동"
},
"선형 결합": {
"목적": "입력 특징들의 새로운 조합 생성",
"효과": "복잡한 패턴 포착 준비",
"수식": "h = xW₁  + b₁ "
},
"정보 분산": {
"목적": "정보를 더 많은 뉴런에 분산",
"효과": "병렬 처리와 특화된 학습 가능",
"장점": "과적합 방지와 일반화 성능 향상"
}
```

```
}

return transformation_effects
```

2. ReLU 활성화 함수의 중요성

```
import torch

import torch.nn.functional as F

import matplotlib.pyplot as plt

import numpy as np


def  analyze_relu_activation():

"""ReLU 활성화 함수의 특성과 효과 분석"""

# ReLU 함수 정의

def  relu(x):

return torch.max(torch.zeros_like(x), x)

# 입력 범위 생성

x = torch.linspace(-3, 3, 1000)

y_relu = F.relu(x)

# ReLU의 특성

relu_properties = {

"비선형성": {

"설명": "선형 변환만으로는 불가능한 복잡한 함수 근사",

"효과": "모델의 표현력 대폭 증가",
```

```
"수식": "f(x) = max(0, x)"
},
"희소성(Sparsity)": {
"설명": "음수 입력을 0으로 만들어 희소한 표현 생성",
"효과": "계산 효율성과 해석 가능성 향상",
"장점": "과적합 방지와 일반화 성능 개선"
},
"그래디언트 특성": {
"설명": "양수 영역에서 그래디언트가 1로 일정",
"효과": "그래디언트 소실 문제 완화",
"장점": "깊은 네트워크에서 안정적인 학습"
},
"계산 효율성": {
"설명": "단순한 max 연산으로 빠른 계산",
"효과": "훈련과 추론 속도 향상",
"구현": "하드웨어 최적화 용이"
}
}
return relu_properties


def  compare_activation_functions():
"""다양한 활성화 함수 비교"""
x = torch.linspace(-3, 3, 1000)
```

```
activations = {
"ReLU": F.relu(x),
"GELU": F.gelu(x), # 최신 트랜스포머에서 사용
"Swish": x * torch.sigmoid(x),
"Tanh": torch.tanh(x)
}
comparison = {
"ReLU": {
"장점": ["단순함", "빠른 계산", "희소성"],
"단점": ["Dead ReLU 문제", "음수 영역 정보 손실"],
"사용": "원본 트랜스포머"
},
"GELU": {
"장점": ["부드러운 곡선", "확률적 해석", "더 나은 성능"],
"단점": ["복잡한 계산", "약간 느림"],
"사용": "BERT, GPT 등 최신 모델"
}
}
return comparison
```

3. 두 번째 선형 변환: 차원 축소

```
def  second_linear_transformation():
```

```
"""
두 번째 선형 변환의 역할과 효과
d_ff → d_model (원래 차원으로 복원)
예: 2048 → 512
"""
transformation_effects = {
"차원 축소": {
"목적": "확장된 표현을 원래 차원으로 압축",
"효과": "중요한 정보만 선별하여 유지",
"비유": "넓은 방에서 얻은 정보를 정리하여 원래 방으로"
},
"정보 압축": {
"목적": "고차원 특징을 저차원으로 요약",
"효과": "노이즈 제거와 핵심 정보 추출",
"수식": "output = h₁ W₂  + b₂ "
},
"잔차 연결 준비": {
"목적": "입력과 동일한 차원으로 맞춤",
"효과": "잔차 연결(residual connection) 가능",
"중요성": "그래디언트 흐름과 학습 안정성"
}
}
```

```
return transformation_effects
```

차원 확장과 축소의 의미와 효과

1. 차원 확장의 이론적 배경

```
def dimension_expansion_theory():
"""차원 확장의 이론적 근거와 실제 효과"""
# 표현 용량(Representational Capacity) 분석
def analyze_capacity():
d_model = 512
d_ff = 2048
# 선형 변환의 표현 용량
linear_capacity = d_model * d_ff # 가중치 매개변수 수
# 비선형 활성화 후 표현 가능한 함수 복잡도
nonlinear_capacity = "무한대 (ReLU 네트워크의 보편 근사 정리)"
return {
"선형 용량": linear_capacity,
"비선형 용량": nonlinear_capacity,
"실제 효과": "복잡한 패턴과 관계 학습 가능"
}
# 정보 처리 관점
def information_processing_view():
```

```
return {
"압축 → 확장 → 압축": {
"단계1": "입력 정보를 고차원 공간으로 투영",
"단계2": "고차원에서 복잡한 변환 수행",
"단계3": "변환된 정보를 원래 차원으로 압축",
"효과": "정보 손실 최소화하면서 복잡한 변환 수행"
}
}
return analyze_capacity(), information_processing_view()
```

2. 4배 확장 비율의 근거

```
def  expansion_ratio_analysis():
"""왜 d_ff = 4 × d_model인가?"""
ratios_comparison = {
"2배 확장": {
"장점": ["적은 매개변수", "빠른 계산"],
"단점": ["제한된 표현력", "복잡한 패턴 학습 어려움"],
"성능": "기본 성능, 단순한 태스크에 적합"
},
"4배 확장": {
"장점": ["균형잡힌 성능", "충분한 표현력", "안정적 학습"],
"단점": ["중간 수준의 계산 비용"],
```

```
"성능": "최적 성능, 대부분 태스크에 적합"
},
"8배 확장": {
"장점": ["매우 높은 표현력", "복잡한 패턴 학습"],
"단점": ["높은 계산 비용", "과적합 위험", "메모리 사용량 증가"],
"성능": "복잡한 태스크에서 약간의 성능 향상"
}
}
# 실험적 근거
empirical_evidence = {
"원본 논문": "Attention Is All You Need에서 4배가 최적으로 확인",
"후속 연구": "다양한 비율 실험 결과 4배가 성능-효율성 균형점",
"실무 적용": "대부분의 트랜스포머 모델이 4배 비율 채택"
}
return ratios_comparison, empirical_evidence
```

3. 메모리와 계산 복잡도 분석

```
def complexity_analysis():
"""피드포워드 네트워크의 복잡도 분석"""
def calculate_complexity(batch_size, seq_len, d_model, d_ff):
# 매개변수 수
params_w1 = d_model * d_ff
```

```
params_b1 = d_ff

params_w2 = d_ff * d_model

params_b2 = d_model

total_params = params_w1 + params_b1 + params_w2 + params_b2

# 계산 복잡도 (FLOPs)

flops_linear1 = batch_size * seq_len * d_model * d_ff

flops_relu = batch_size * seq_len * d_ff

flops_linear2 = batch_size * seq_len * d_ff * d_model

total_flops = flops_linear1 + flops_relu + flops_linear2

# 메모리 사용량 (중간 활성화)

memory_input = batch_size * seq_len * d_model

memory_hidden = batch_size * seq_len * d_ff

memory_output = batch_size * seq_len * d_model

peak_memory = memory_input + memory_hidden + memory_output

return {

"매개변수 수": total_params,

"계산 복잡도 (FLOPs)": total_flops,

"피크 메모리": peak_memory

}

# 예시 계산 (BERT-base 크기)

example = calculate_complexity(

batch_size=32,

seq_len=128,
```

```
d_model=512,
d_ff=2048
)
return example
```

PositionwiseFeedForward 클래스 구현

1. 기본 구현

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class PositionwiseFeedForward(nn.Module):
"""
포지션별 피드포워드 네트워크
구조:
입력 → Linear(d_model → d_ff) → ReLU → Dropout → Linear(d_ff → d_model) → 출력
Args:
d_model (int): 모델의 차원 (입력/출력 차원)
d_ff (int): 피드포워드 네트워크의 내부 차원 (보통 d_model의 4배)
dropout (float): 드롭아웃 비율 (기본값: 0.1)
activation (str): 활성화 함수 종류 ('relu', 'gelu', 'swish')
```

```
"""
def __init__(self, d_model, d_ff, dropout=0.1, activation='relu'):
super(PositionwiseFeedForward, self).__init__()
self.d_model = d_model
self.d_ff = d_ff
self.activation_name = activation
# 첫 번째 선형 변환: d_model → d_ff (차원 확장)
self.linear1 = nn.Linear(d_model, d_ff)
# 두 번째 선형 변환: d_ff → d_model (차원 축소)
self.linear2 = nn.Linear(d_ff, d_model)
# 드롭아웃 (정규화)
self.dropout = nn.Dropout(dropout)
# 활성화 함수 선택
self.activation = self._get_activation(activation)
def _get_activation(self, activation):
"""활성화 함수 선택"""
if activation == 'relu':
return F.relu
elif activation == 'gelu':
return F.gelu
elif activation == 'swish':
return lambda x: x * torch.sigmoid(x)
else:
```

```
raise ValueError(f"지원하지 않는 활성화 함수: {activation}")

def  forward(self, x):

"""

순전파

Args:

x: [batch_size, seq_len, d_model] - 입력 텐서

Returns:

output: [batch_size, seq_len, d_model] - 출력 텐서

"""

# 입력 형태 확인

batch_size, seq_len, d_model = x.size()

assert d_model == self.d_model, f"입력 차원 불일치: {d_model} != {self.d_model}"

# 1. 첫 번째 선형 변환 (차원 확장)

# [batch_size, seq_len, d_model] → [batch_size, seq_len, d_ff]

hidden = self.linear1(x)

# 2. 활성화 함수 적용 (비선형성 도입)

hidden = self.activation(hidden)

# 3. 드롭아웃 적용 (정규화)

hidden = self.dropout(hidden)

# 4. 두 번째 선형 변환 (차원 축소)

# [batch_size, seq_len, d_ff] → [batch_size, seq_len, d_model]

output = self.linear2(hidden)

return output
```

```
def extra_repr(self):
"""모델 정보 출력용"""
return f'd_model={self.d_model}, d_ff={self.d_ff}, activation={self.activation_
```

2. 고급 구현 (최적화 포함)

```
class OptimizedPositionwiseFeedForward(nn.Module):
"""
최적화된 피드포워드 네트워크
추가 기능:
- 가중치 초기화
- 그래디언트 체크포인팅
- 혼합 정밀도 지원
- 메모리 효율적 구현
"""
def __init__(self, d_model, d_ff, dropout=0.1, activation='relu',
use_bias=True, init_method='xavier'):
super(OptimizedPositionwiseFeedForward, self).__init__()
self.d_model = d_model
self.d_ff = d_ff
self.activation_name = activation
self.use_bias = use_bias
# 선형 변환 레이어
```

```
self.linear1 = nn.Linear(d_model, d_ff, bias=use_bias)

self.linear2 = nn.Linear(d_ff, d_model, bias=use_bias)

# 드롭아웃

self.dropout = nn.Dropout(dropout)

# 활성화 함수

self.activation = self._get_activation(activation)

# 가중치 초기화

self._initialize_weights(init_method)

def _get_activation(self, activation):

"""활성화 함수 선택 (인플레이스 연산 최적화)"""

if activation == 'relu':

return nn.ReLU(inplace=True)

elif activation == 'gelu':

return nn.GELU()

elif activation == 'swish':

return nn.SiLU() # Swish와 동일하지만 더 효율적

else:

raise ValueError(f"지원하지 않는 활성화 함수: {activation}")

def _initialize_weights(self, method):

"""가중치 초기화"""

for module in [self.linear1, self.linear2]:

if method == 'xavier':

nn.init.xavier_uniform_(module.weight)
```

```
elif method == 'kaiming':
nn.init.kaiming_uniform_(module.weight, nonlinearity='relu')
elif method == 'normal':
nn.init.normal_(module.weight, std=0.02)
if  self.use_bias and module.bias is not None:
nn.init.constant_(module.bias, 0)
def  forward(self, x):
"""메모리 효율적인 순전파"""
def  _forward_impl(x):
# 첫 번째 변환
x = self.linear1(x)
x = self.activation(x)
x = self.dropout(x)
# 두 번째 변환
x = self.linear2(x)
return x
# 훈련 시 그래디언트 체크포인팅 사용 (메모리 절약)
if  self.training and x.requires_grad:
return torch.utils.checkpoint.checkpoint(_forward_impl, x)
else:
return _forward_impl(x)
```

3. 다양한 활성화 함수 지원

```python
class FlexibleFeedForward(nn.Module):
"""
다양한 활성화 함수를 지원하는 피드포워드 네트워크
"""
def __init__(self, d_model, d_ff, dropout=0.1, activation='relu'):
super(FlexibleFeedForward, self).__init__()
self.linear1 = nn.Linear(d_model, d_ff)
self.linear2 = nn.Linear(d_ff, d_model)
self.dropout = nn.Dropout(dropout)
# 활성화 함수 매핑
self.activations = {
'relu': nn.ReLU(),
'gelu': nn.GELU(),
'swish': nn.SiLU(),
'mish': nn.Mish(),
'leaky_relu': nn.LeakyReLU(0.1),
'elu': nn.ELU(),
'selu': nn.SELU()
}
self.activation = self.activations.get(activation, nn.ReLU())
def forward(self, x):
return self.linear2(self.dropout(self.activation(self.linear1(x))))
```

```python
def  compare_activations_performance():
"""다양한 활성화 함수의 성능 비교"""
activations = ['relu', 'gelu', 'swish', 'mish']
d_model, d_ff = 512, 2048
batch_size, seq_len = 16, 128
results = {}
for activation in activations:
model = FlexibleFeedForward(d_model, d_ff, activation=activation)
x = torch.randn(batch_size, seq_len, d_model)
# 순전파 시간 측정
import time
start_time = time.time()
with torch.no_grad():
for _ in  range(100):
_ = model(x)
end_time = time.time()
results[activation] = {
'time_per_forward': (end_time - start_time) / 100,
'parameters': sum(p.numel() for p in model.parameters()),
'output_stats': {
'mean': model(x).mean().item(),
'std': model(x).std().item()
```

```
}
}
return results
```

완전한 사용 예시와 성능 분석

1. 기본 사용 예시

```
def  demonstrate_feedforward():
"""피드포워드 네트워크 사용 예시"""
# 하이퍼파라미터 설정
batch_size = 8
seq_len = 64
d_model = 512
d_ff = 2048
# 모델 생성
ff_network = PositionwiseFeedForward(
d_model=d_model,
d_ff=d_ff,
dropout=0.1,
activation='relu'
)
# 입력 데이터 생성 (보통 어텐션 레이어의 출력)
```

```
input_tensor = torch.randn(batch_size, seq_len, d_model)

print(f"입력 크기: {input_tensor.shape}")

print(f"모델 매개변수 수: {sum(p.numel() for p in ff_network.parameters()):,}")

# 순전파

output = ff_network(input_tensor)

print(f"출력 크기: {output.shape}")

print(f"입력-출력 차원 일치: {input_tensor.shape == output.shape}")

# 통계 정보

print(f"\n입력 통계:")

print(f" 평균: {input_tensor.mean():.4f}")

print(f" 표준편차: {input_tensor.std():.4f}")

print(f"\n출력 통계:")

print(f" 평균: {output.mean():.4f}")

print(f" 표준편차: {output.std():.4f}")

return ff_network, input_tensor, output



# 실행

model, input_data, output_data = demonstrate_feedforward()
```

2. 트랜스포머 레이어에서의 통합 사용

```
class TransformerLayerWithFFN(nn.Module):

"""
```

```
피드포워드 네트워크가 포함된 완전한 트랜스포머 레이어

"""

def  __init__(self, d_model, num_heads, d_ff, dropout=0.1):

super(TransformerLayerWithFFN, self).__init__()

# 멀티헤드 어텐션 (이전 문서에서 구현)

self.self_attention = MultiHeadAttention(d_model, num_heads, dropout)

# 피드포워드 네트워크

self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

# 레이어 정규화

self.norm1 = nn.LayerNorm(d_model)

self.norm2 = nn.LayerNorm(d_model)

# 드롭아웃

self.dropout = nn.Dropout(dropout)

def  forward(self, x, mask=None):

"""

트랜스포머 레이어 순전파

구조:

1. 멀티헤드 어텐션 + 잔차 연결 + 레이어 정규화

2. 피드포워드 네트워크 + 잔차 연결 + 레이어 정규화

"""

# 1. 어텐션 서브레이어

attention_output, attention_weights = self.self_attention(x, x, x, mask)

attention_output = self.dropout(attention_output)
```

```
x = self.norm1(x + attention_output) # 잔차 연결 + 정규화

# 2. 피드포워드 서브레이어

ff_output = self.feed_forward(x)

ff_output = self.dropout(ff_output)

output = self.norm2(x + ff_output) # 잔차 연결 + 정규화

return output, attention_weights



def  test_complete_layer():

"""완전한 트랜스포머 레이어 테스트"""

# 설정

batch_size, seq_len = 4, 32

d_model, num_heads, d_ff = 512, 8, 2048

# 레이어 생성

layer = TransformerLayerWithFFN(d_model, num_heads, d_ff)

# 입력 데이터

x = torch.randn(batch_size, seq_len, d_model)

# 순전파

output, attention_weights = layer(x)

print(f"완전한 레이어 테스트:")

print(f"입력 크기: {x.shape}")

print(f"출력 크기: {output.shape}")

print(f"어텐션 가중치 크기: {attention_weights.shape}")
```

```
return layer, output
```

3. 성능 벤치마크와 최적화

```
def  benchmark_feedforward_variants():
"""다양한 피드포워드 구현의 성능 비교"""
import time
import torch.profiler
# 테스트 설정
batch_size, seq_len = 32, 128
d_model, d_ff = 512, 2048
num_iterations = 100
# 다양한 구현 버전
models = {
'basic': PositionwiseFeedForward(d_model, d_ff),
'optimized': OptimizedPositionwiseFeedForward(d_model, d_ff),
'gelu': PositionwiseFeedForward(d_model, d_ff, activation='gelu')
}
# 입력 데이터
x = torch.randn(batch_size, seq_len, d_model)
results = {}
for name, model in models.items():
model.eval()
```

```
# 워밍업
with torch.no_grad():
for _ in  range(10):
_ = model(x)
# 시간 측정
start_time = time.time()
with torch.no_grad():
for _ in  range(num_iterations):
output = model(x)
end_time = time.time()
# 메모리 사용량 측정
torch.cuda.empty_cache() if torch.cuda.is_available() else None
memory_before = torch.cuda.memory_allocated() if torch.cuda.is_available() else
with torch.no_grad():
_ = model(x)
memory_after = torch.cuda.memory_allocated() if torch.cuda.is_available() else
results[name] = {
'avg_time': (end_time - start_time) / num_iterations,
'memory_usage': memory_after - memory_before,
'parameters': sum(p.numel() for p in model.parameters()),
'output_mean': output.mean().item(),
'output_std': output.std().item()
}
```

```
return results


def  analyze_dimension_effects():
"""차원 비율이 성능에 미치는 영향 분석"""
d_model = 512
expansion_ratios = [1, 2, 4, 8, 16]
results = {}
for ratio in expansion_ratios:
d_ff = d_model * ratio
model = PositionwiseFeedForward(d_model, d_ff)
# 매개변수 수 계산
params = sum(p.numel() for p in model.parameters())
# 메모리 사용량 추정
batch_size, seq_len = 16, 64
input_memory = batch_size * seq_len * d_model * 4  # float32
hidden_memory = batch_size * seq_len * d_ff * 4
total_memory = input_memory + hidden_memory
results[f'{ratio}x'] = {
'parameters': params,
'memory_mb': total_memory / (1024 * 1024),
'flops': batch_size * seq_len * (d_model * d_ff + d_ff * d_model)
}
```

```
return results
```

실무 적용 고려사항

1. 하이퍼파라미터 선택 가이드

```
def hyperparameter_guidelines():
"""피드포워드 네트워크 하이퍼파라미터 선택 가이드"""
guidelines = {
"d_ff 선택": {
"일반 원칙": "d_ff = 4 × d_model",
"작은 모델": "d_ff = 2 × d_model (리소스 제약시)",
"큰 모델": "d_ff = 4-8 × d_model (성능 중시시)",
"고려사항": [
"메모리 제약",
"계산 시간 요구사항",
"태스크 복잡도",
"데이터셋 크기"
]
},
"드롭아웃 비율": {
"기본값": "0.1",
"작은 데이터": "0.2-0.3 (과적합 방지)",
```

```
"큰 데이터": "0.05-0.1 (성능 최적화)",
"주의사항": "너무 높으면 언더피팅 위험"
},
"활성화 함수": {
"ReLU": "기본 선택, 빠르고 안정적",
"GELU": "최신 모델에서 선호, 약간 더 나은 성능",
"Swish": "일부 태스크에서 우수한 성능",
"선택 기준": "성능 vs 계산 효율성 트레이드오프"
}
}
return guidelines


def  memory_optimization_tips():
"""메모리 최적화 팁"""
tips = {
"그래디언트 체크포인팅": {
"설명": "중간 활성화를 저장하지 않고 필요시 재계산",
"장점": "메모리 사용량 대폭 감소",
"단점": "계산 시간 약간 증가",
"적용": "메모리 제약이 심한 환경"
},
"혼합 정밀도": {
"설명": "FP16과 FP32를 혼합하여 사용",
```

```
"장점": "메모리 절약 + 속도 향상",
"단점": "수치 안정성 주의 필요",
"적용": "최신 GPU에서 권장"
},
"인플레이스 연산": {
"설명": "새로운 텐서 생성 없이 기존 텐서 수정",
"장점": "메모리 사용량 감소",
"단점": "그래디언트 계산에 주의 필요",
"적용": "ReLU 등 활성화 함수"
}
}
return tips
```

2. 디버깅과 모니터링

```
class DebuggableFeedForward(PositionwiseFeedForward):
"""디버깅 기능이 포함된 피드포워드 네트워크"""
def __init__(self, d_model, d_ff, dropout=0.1, activation='relu', debug=False):
super().__init__(d_model, d_ff, dropout, activation)
self.debug = debug
self.stats = {}
def forward(self, x):
if self.debug:
```

```
self.stats['input'] = {
'shape': x.shape,
'mean': x.mean().item(),
'std': x.std().item(),
'min': x.min().item(),
'max': x.max().item()
}
# 첫 번째 선형 변환
hidden = self.linear1(x)
if  self.debug:
self.stats['after_linear1'] = {
'shape': hidden.shape,
'mean': hidden.mean().item(),
'std': hidden.std().item(),
'zero_ratio': (hidden == 0).float().mean().item()
}
# 활성화 함수
hidden = self.activation(hidden)
if  self.debug:
self.stats['after_activation'] = {
'mean': hidden.mean().item(),
'std': hidden.std().item(),
'zero_ratio': (hidden == 0).float().mean().item(),
```

```
'positive_ratio': (hidden > 0).float().mean().item()
}
# 드롭아웃
hidden = self.dropout(hidden)
# 두 번째 선형 변환
output = self.linear2(hidden)
if  self.debug:
self.stats['output'] = {
'shape': output.shape,
'mean': output.mean().item(),
'std': output.std().item()
}
return output
def  get_stats(self):
"""디버깅 통계 반환"""
return  self.stats
def  reset_stats(self):
"""통계 초기화"""
self.stats = {}


def  debug_feedforward_behavior():
"""피드포워드 네트워크 동작 분석"""
# 디버깅 모드로 모델 생성
```

```
model = DebuggableFeedForward(512, 2048, debug=True)

# 테스트 입력

x = torch.randn(4, 32, 512)

# 순전파

output = model(x)

# 통계 출력

stats = model.get_stats()

print("피드포워드 네트워크 동작 분석:")

print("=" * 50)

for stage, stat in stats.items():

print(f"\n{stage}:")

for key, value in stat.items():

if  isinstance(value, float):

print(f" {key}: {value:.4f}")

else:

print(f" {key}: {value}")

return stats
```

핵심 개념 정리

1. 피드포워드 네트워크의 핵심 역할

- **개별적 변환**: 각 토큰을 독립적으로 처리하여 표현력 향상
- **비선형성 도입**: ReLU 활성화를 통한 복잡한 함수 근사 능력

- **차원 확장-축소**: 고차원 공간에서의 복잡한 변환 후 원래 차원으로 압축
- **어텐션과 상호보완**: 어텐션의 관계 모델링과 FFN의 개별 변환이 결합

2. 구현 시 주요 고려사항

- **차원 비율**: d_ff = 4 × d_model이 일반적으로 최적
- **활성화 함수**: ReLU(기본) vs GELU(성능) vs Swish(특수 용도)
- **정규화**: 드롭아웃을 통한 과적합 방지
- **메모리 최적화**: 그래디언트 체크포인팅, 혼합 정밀도 등

3. 성능 최적화 전략

- **계산 효율성**: 인플레이스 연산, 효율적인 활성화 함수
- **메모리 효율성**: 체크포인팅, 적절한 배치 크기
- **하드웨어 최적화**: GPU 메모리 패턴, 텐서 코어 활용

다음 단계

피드포워드 네트워크를 이해했다면, 다음 문서에서는 **레이어 정규화와 잔차 연결**을 자세히 살펴보겠습니다. 이들은 트랜스포머의 안정적인 학습을 가능하게 하는 핵심 기법들입니다.

네비게이션

- **이전**: 디코더 아키텍처
- **다음**: 레이어 정규화와 잔차 연결
- **목차**: README.md
- **전체 목록**: 네비게이션

관련 문서:

- 기초: 트랜스포머 개요
- 구성 요소: 인코더 아키텍처
- 구성 요소: 디코더 아키텍처